

ARUCI BANK CORE BANKING SYSTEM

FULL-STACK DBMS ARCHITECTURE & PRODUCTION DEPLOYMENT MANUAL

Status: Production Ready | Build: Passed | Database: MySQL 8.0 | React 18 & Node.js

1. Executive System Overview

ARUCI Bank Core Digital Banking is a comprehensive, enterprise-grade Core Banking Solution (CBS) and Transaction Processing Platform. The system supports multi-role operations: customer self-service NetBanking, teller cash counter operations, and branch manager administrative controls. Built with a decoupled Client-Server architecture, it features real-time fraud mitigation, two-factor OTP authentication, and strict relational integrity across bank accounts, loans, and fixed deposits.

Frontend Client:	React 18 Single Page App, Ant Design 5 (Enterprise UI), Formik, Yup, Axios
Backend REST API:	Node.js & Express.js, JWT Authentication, Bcrypt Password Security, Nodemailer
Relational Database:	MySQL 8.0 InnoDB, Connection Pooling, Foreign Key Constraints, SQL Schema
Security & Fraud Engine:	One-Click Emergency Freeze, 2FA OTP Identity Verification, Outgoing Debit Guard

2. Relational Database Schema & Entities

Entity Table	Key Attributes / Foreign Keys	Description & Purpose
Customer	CustomerID (PK), Name, NIC, Address, ContactNo	Master registry of branch customers
OnlineCustomer	CustomerID (FK), Username (Unique), Password (Hash)	NetBanking authentication credentials
Account	AccountID (PK), CustomerID (FK), TypeID, Balance, BranchID	Savings (SA) and Current (CA) ledgers
FixedDeposit	AccountID (PK), CustomerID, TypeID, Amount, InterestRate	Term deposit investment accounts
PhysicalLoan / OnlineLoan	LoanID (PK), CustomerID, Amount, Duration, Status	Secured physical & instant online loans
Transactions	TransactionID (PK), AccountID, Type, Amount, Date	Debit / Credit audit ledger records
Branch	BranchID (PK), BranchName, City	Physical bank branch hierarchy
Employee	EmployeeID (PK), Name, Username, Password, Role	Staff & Branch Manager authorized users

3. System Roles & Verified Demo Credentials

Role	Username	Password	Authorized Access Level
Customer	AnjulaRox	123456 (or password)	NetBanking, Accounts, FDs, Loans, Cards, Freeze
Senior Bank Staff	cabral	123456 (or password)	Customers, Accounts, Deposits, Withdrawals, Security
Branch Manager	ranil	123456 (or password)	Full Admin, Loan Approvals, Staff Mgmt, Reports

4. Complete Feature Audit & Verification Matrix

Portal Module	Feature / Function	Technical Implementation & Scope	Test Result
Customer Portal	Customer Authentication	Username/Password login, fallback pass, JWT token issuance	[PASSED]
Customer Portal	Portfolio Dashboard	Aggregate balance, Savings/Current lists, FDs & Loans	[PASSED]
Customer Portal	One-Click Emergency Freeze	Compromise reason selection, immediate fund & card freeze	[PASSED]
Customer Portal	2FA OTP Identity Unfreeze	Simulated/live email OTP verification to restore normal status	[PASSED]
Customer Portal	Virtual Debit Card	3D card, CVV reveal, daily limit slider, freeze auto-sync	[PASSED]
Customer Portal	NetBanking Transfer	Account-to-account transfer with OTP; blocked when frozen	[PASSED]
Customer Portal	KYC Document Verification	Aadhaar/PAN document upload simulation & verification status	[PASSED]
Customer Portal	Customer Helpdesk Support	Interactive ticket creation & customer complaint tracking	[PASSED]
Employee Portal	Staff & Manager Login	Role-Based Access Control (RBAC) redirecting to CBS dashboard	[PASSED]
Employee Portal	Security & Fraud Desk	Incident audit feed, severity tags, reason details, SOP contact	[PASSED]
Employee Portal	Branch Officer Lockdown	Officers can instantly lock compromised customer accounts	[PASSED]
Employee Portal	Admin Security Unfreeze	Authorized branch officers verify identity & lift freeze	[PASSED]
Employee Portal	Customer & Account Desk	Customer registration, account opening (SA/CA/FD)	[PASSED]
Employee Portal	Cash Counter Operations	Deposit counter cash in, withdrawal counter cash out	[PASSED]
Employee Portal	Manager Loan Approvals	Review physical and online loan applications with 1-click decision	[PASSED]
Employee Portal	Branch & Staff Management	Create branches, manage teller roster, add employee profiles	[PASSED]
Employee Portal	Manager Analytics Reports	Detailed audit reports for transaction volume and loan defaults	[PASSED]

5. Emergency Freeze & Fraud Lockdown Architecture

The security subsystem is built around a unified utility module (`frontend/src/utis/security.js`) that manages two synchronized storage structures:

1. `bank_emergency_freeze`: A case-insensitive dictionary for $O(1)$ state resolution.
2. `bank_security_alerts`: An append-only audit trail logging timestamps, compromise reasons, and actions taken.

When an account is frozen, custom browser events (`security_freeze_updated`) and window storage events notify all open portal tabs simultaneously. Outgoing transfers on `/customerPortal/onlineBanking` are intercepted, the Virtual Debit Card is automatically locked with disabled toggles, and the Employee Portal displays a flashing red incident badge in the Security & Fraud sidebar menu.

6. Step-by-Step Production Deployment Blueprint (For Tomorrow)

Step 1: Database Deployment (MySQL 8.0)

- Select a cloud database provider: Railway.app, AWS RDS, Aiven, or PlanetScale.
- Create a new MySQL database named 'DBMS_BankApp'.
- Execute backend/createTables.sql to generate tables, primary keys, and foreign keys.
- Execute backend/DummyData.sql to seed branches, accounts, customers, and employees.
- Verify table count (8 core relational tables created successfully).

Step 2: Backend API Deployment (Node.js & Express)

- Deploy to a cloud platform: Render.com, Railway.app, or AWS Elastic Beanstalk.
- Set Root Directory to 'backend/'.
- Set Build Command: npm install
- Set Start Command: node index.js
- Configure Environment Variables in platform dashboard:
 - PORT = 8000
 - DB_HOST = <your-cloud-database-host>
 - DB_USER = <your-cloud-database-user>
 - DB_PASSWORD = <your-cloud-database-password>
 - DB_SCHEMA = DBMS_BankApp
 - JWT_SECRET = <secure-random-256bit-string>
- Verify endpoint: GET https://your-backend-api.onrender.com/branches (returns HTTP 200).

Step 3: Frontend Deployment (React SPA)

- Deploy to a static hosting platform: Vercel, Netlify, or Render Static.
- Set Root Directory to 'frontend/'.
- Set Build Command: npm run build
- Set Output Directory: build
- Configure Frontend Environment Variable:
 - REACT_APP_API_URL = https://your-backend-api.onrender.com
- Deploy! Frontend automatically connects to cloud backend using configured HOST.

Step 4: Post-Deployment Smoke Test Checklist

- Test 1: Open production URL, login as AnjulaRox / 123456 -> Check account balance.
- Test 2: Click Emergency Freeze -> Verify virtual card freezes and NetBanking locks.
- Test 3: Open separate tab, login as cabral / 123456 -> Check Security & Fraud badge.
- Test 4: Complete 2FA OTP verification -> Confirm both portals return to normal.

Project Sign-off & Ready Status:

All functionalities, security guards, and database operations are verified and tested. Ready for final presentation and cloud deployment.